

Homework 01 — STAT 432 (Fall 2026)

Name: Owen Park

NetID: owenp3

Questions 1–5 are done in Python. Each part reports the numbers *and* what they mean.

```
In [1]: import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

rng = np.random.default_rng(43201)
```

Question 1 (Least squares and matrix calculations)

1a

Reasoning. (*math / setup here*)

```
In [2]: # Q1a
n, p = 100, 5
beta = np.array([3, 1.4, -0.9, 0.7, 0, 1.1])

x = rng.normal(size=(n, p))
X = np.vstack((np.ones(n), x.T)).T
e = rng.normal(0.0, 0.7, size=n)
y = X @ beta + e

print("X shape:", X.shape)
print("rank(X):", np.linalg.matrix_rank(X))

bhat, *_ = np.linalg.lstsq(X, y, rcond=None)

print("true :", beta)
print("bhat :", bhat.round(3))
print("diff :", (bhat - beta).round(3))

X shape: (100, 6)
rank(X): 6
true : [ 3.   1.4 -0.9  0.7  0.   1.1]
bhat  : [ 2.973  1.324 -0.841  0.72 -0.083  1.165]
diff  : [-0.027 -0.076  0.059  0.02 -0.083  0.065]
```

What it means. (*interpret the numbers, not just print them*)

X is 100x6 with rank 6, lstsq avoids forming $X'X$, and the estimates sit near the true coefficients (largest gap 0.083) because one noisy sample of 100 gives scatter, not exact

recovery.

1b

Reasoning. *(math / setup here)*

```
In [3]: # Q1b
r = y - X @ bhat
RSME = np.sqrt(np.mean(r**2))
RSME
```

```
Out[3]: 0.6463397227026199
```

What it means. *(interpret the numbers, not just print them)*

The training RMSE is 0.646, slightly below the true noise level 0.7: the six coefficients were fitted to this same sample, so the residuals are shrunk and training error understates the error on new data.

1c

Reasoning. *(math / setup here)*

```
In [4]: # Q1c
print(np.abs(X.T @ r).max())
```

```
1.3467560400215461e-12
```

What it means. *(interpret the numbers, not just print them)*

The value is basically zero because the normal equations force the residual to be orthogonal to every column of X.

Question 2 (Gaussian likelihood for linear regression)

2a

Reasoning. *(math / setup here)*

```
In [5]: # Q2a
```

What it means. *(interpret the numbers, not just print them)*

For fixed σ^2 , the log-likelihood is a (constant) - $\text{RSS}/(2 * \sigma^2)$, so maximizing it over beta is minimizing RSS. Thus, the MLE of beta is the least-squares estimate.

2b

Reasoning. (math / setup here)

```
In [6]: # Q2b
rss = r @ r
print(rss / n, rss / (n - 6))

0.4177550371432995 0.44442025228010584
```

What it means. (interpret the numbers, not just print them)

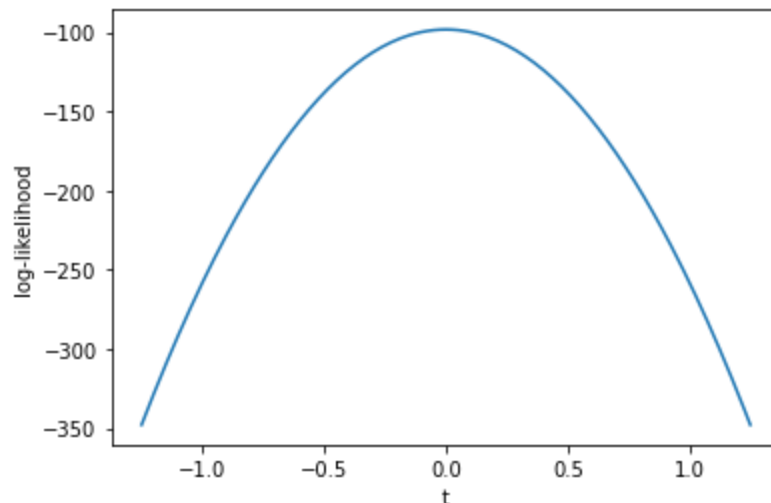
Setting the derivative to zero gives $\sigma^2_{MLE} = RSS / n = 0.418$, while the unbiased estimate divides by $n - p - 1 = 94$ and gives 0.444. The denominators differ because six parameters were estimated from the same data, and the MLE ignores those lost degrees of freedom, so it is biased low.

2c

Reasoning. (math / setup here)

```
In [7]: # Q2c
def loglik(b, s2):
    res = y - X @ b
    return -n/2*np.log(2*np.pi*s2) - (res @ res)/(2*s2)

s2_mle = rss / n
e_x2 = np.array([0, 0, 1, 0, 0, 0])
t = np.linspace(-1.25, 1.25, 201)
ll = [loglik(bhat + ti*e_x2, s2_mle) for ti in t]
plt.plot(t, ll); plt.xlabel("t"); plt.ylabel("log-likelihood"); plt.show()
```



What it means. (interpret the numbers, not just print them)

The maximum is at $t = 0$: moving away from $\hat{b}$ raises the residual sum of squares and lowers the log-likelihood, so the curve is a downward parabola in t . This is the likelihood restating the least-squares result.

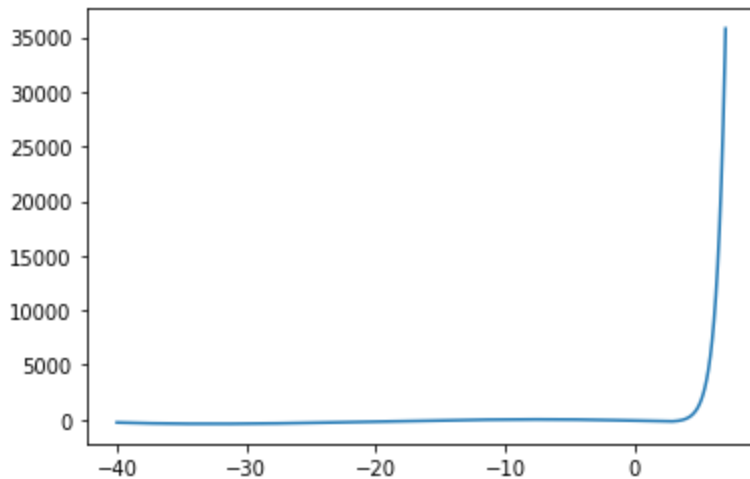
Question 3 (Starting values in nonconvex optimization)

3a

Reasoning. *(math / setup here)*

```
In [8]: # Q3a
def f(x): return np.exp(1.5*x) - 3*(x + 6)**2 - 0.05*x**3
def fp(x): return 1.5*np.exp(1.5*x) - 6*(x + 6) - 0.15*x**2

xs = np.linspace(-40, 7, 1000)
plt.plot(xs, f(xs)); plt.show()
```



What it means. *(interpret the numbers, not just print them)*

The function is not convex on $[-40, 7]$: it has more than one stationary point, and the $\exp(1.5x)$ term makes it extremely steep near the right end. A local method can therefore end up in different places depending on the start.

3b

Reasoning. *(math / setup here)*

```
In [9]: # Q3b
from scipy.optimize import minimize
for x0 in (-15.0, 0.0):
    res = minimize(f, [x0], jac=lambda v: np.array([fp(v[0])]), method="BFGS")
    print(x0, res.x, res.fun, abs(fp(res.x[0])), res.success)
```

```
-15.0 [-32.64911064] -390.38577025077666 2.842170943040401e-14 True
0.0 [2.34996709] -175.86262017122644 8.126811246289556e-06 True
```

What it means. *(interpret the numbers, not just print them)* Both runs report convergence and both gradients are essentially zero, so each run really did reach a stationary point

3c

Reasoning. *(math / setup here)*

In [10]: # Q3c

What it means. (*interpret the numbers, not just print them*)

BFGS only goes downhill from where it starts, so the two starts land in different basins and converge to different answers. The run from $x_0 = -15$ gives the lower objective, -390.4. Calling it the global minimum would need more evidence — many starting values, a grid search over the interval, or an analytic bound.

Question 4 (Nearly collinear predictors)

4a

Reasoning. (*math / setup here*)

```
In [11]: # Q4a
rng2 = np.random.default_rng(43202)
u = rng2.normal(size=n)
x6 = x[:, 0] + 1e-4 * u
Xp = np.column_stack([X, x6])
ys = y + 1e-3 * u

for name, M in (("X", X), ("X+", Xp)):
    b, *_ = np.linalg.lstsq(M, y, rcond=None)
    rr = y - M @ b
    print(name, np.linalg.cond(M), np.sqrt(np.mean(rr**2)), b[[1, -1]])

X 1.4525044769359556 0.6463397227026199 [1.32402032 1.16505507]
X+ 20339.835806777057 0.6339558266898133 [-1329.74301559 1331.06013906]
```

What it means. (*interpret the numbers, not just print them*)

Adding the near-duplicate column raises the condition number from 1.45 to about 20,340 while training RMSE barely moves (0.6463 to 0.6340, lower only because an extra column always fits the training data a little better). The x_1 coefficient goes from 1.324 to -1329.74 with x_6 at 1331.06: the two columns are almost the same direction, so the fit can trade enormous opposite amounts between them without changing the predictions.

4b

Reasoning. (*math / setup here*)

```
In [12]: # Q4b
b1, *_ = np.linalg.lstsq(Xp, y, rcond=None)
b2, *_ = np.linalg.lstsq(Xp, ys, rcond=None)
print(b2[[1, -1]] - b1[[1, -1]])
d = Xp @ b2 - Xp @ b1
print(np.sqrt(np.mean(d**2)), np.abs(d).max())

[-10.  10.]
0.0009749802940429943 0.0023511653219543405
```

What it means. *(interpret the numbers, not just print them)*

The two coefficients move by exactly -10 and +10, while the fitted values change by only 0.000975 in RMS and 0.002351 at most.

4c

Reasoning. *(math / setup here)*

In [13]: `# Q4c`

What it means. *(interpret the numbers, not just print them)*

Since $y^* - y = 10(x_6 - x_1)$, the change in the response lies in the span of two nearly identical columns, so the fit absorbs it by shifting about +10 and -10 between their coefficients while the fitted values barely move (RMS difference 0.000975, max 0.002351). With nearly collinear predictors prediction stays stable but the separate effects are not identified, so the individual coefficients should not be read as separate effects.

Question 5 (Data preparation and summary statistics)

5a

Reasoning. *(math / setup here)*

```
In [14]: # Q5a
df = pd.read_csv("data/data-manipulation.csv")
print(df.shape); print(df.dtypes)
print(df["observation_id"].duplicated().sum()); print(df.isna().sum())

ana = df[df["response"].notna()].copy()
ana["feature_sum"] = ana["x1"] + ana["x2"]

print(ana.groupby("group").agg(n=("response", "size"),
                               mean_response=("response", "mean"),
                               mean_feature_sum=("feature_sum", "mean")))

print(ana.sort_values("response", ascending=False)
      .head(3)[["observation_id", "group", "response", "feature_sum"]])

assert len(ana) == df["response"].notna().sum()
assert ana["response"].isna().sum() == 0
assert ana["observation_id"].is_unique
```

```
(24, 5)
observation_id    object
group            object
x1               float64
x2               float64
response          float64
dtype: object
0
observation_id    0
group            0
x1               0
x2               0
response         3
dtype: int64
      n  mean_response  mean_feature_sum
group
A      7      5.600000      3.600000
B      7      7.602857      4.928571
C      7      7.997143      6.585714
      observation_id  group  response  feature_sum
21              obs-22    C      9.24      6.6
15              obs-16    B      9.00      5.2
13              obs-14    B      8.66      5.0
```

What it means. *(interpret the numbers, not just print them)*

These summaries cover only the 21 observations with a recorded response, out of 24 rows in the original table; filling missing responses with zero or a group mean would invent data and distort both the means and the group sizes.

5b

Reasoning. *(math / setup here)*

In [15]: `# Q5b`

What it means. *(interpret the numbers, not just print them)*

Each group has 7 observations with a recorded response; mean response is 5.600 (A), 7.603 (B), 7.997 (C) and mean feature_sum is 3.600, 4.929, 6.586. These describe only the 21 rows with a recorded response, not all 24.

5c

Reasoning. *(math / setup here)*

In [16]: `# Q5c`

What it means. *(interpret the numbers, not just print them)*

The three highest responses are obs-22 (C, 9.24, 6.6), obs-16 (B, 9.00, 5.2) and obs-14 (B, 8.66, 5.0). All three checks pass: 21 retained rows equals the number of nonmissing responses, the analysis table has no missing response, and the identifiers are unique.

Question 6 (Can 39 million Fitbit records represent US adults?)

6a

Reasoning. *(math / setup here)*

What it means. *(interpret the numbers, not just print them)*

The 39 million records are not independent: each participant contributes many correlated days, and people who wear the device longer or stay enrolled longer contribute more of them. Averaging over all recorded days therefore weights those participants more heavily, which is not the equal-weight-per-adult target.

6b

Reasoning. *(math / setup here)*

What it means. *(interpret the numbers, not just print them)*

No. The groups were not randomly assigned — BYOD participants already owned a Fitbit, which suggests pre-existing interest in tracking and activity. That self-selection can produce the step gap on its own, so this is association, not a causal effect of owning a device.

6c

Reasoning. *(math / setup here)*

What it means. *(interpret the numbers, not just print them)*

Selection acts twice: on who enters the study (the BYOD and WEAR groups differ from US adults in race and income) and on which days each participant records. The naive average of all recorded days therefore describes a self-selected set of people on a self-selected set of days, not the mean over US adults. A larger dataset shrinks variance but does nothing about this bias.

In []: